

Development and Evaluation of a Retail Unit Price Prediction System using MLflow

AIN3009 – Delivering AI Applications with MLOps

University: Bahçeşehir University

Student Name: Masa Bokhary

Student ID: 2287036

Instructor: Gökşin Bakır

Date: May 2026

Table of Contents

Table of Contents.....	<i>Error! Bookmark not defined.</i>
1. Introduction	4
2. Project Objectives.....	4
3. Dataset Description	5
3.1 Dataset Fields	5
3.2 Target Variable.....	5
3.3 Data Characteristics.....	5
4. System Architecture.....	6
4.1 Pipeline Flow	6
4.2 Key Design Decisions	6
5. Data Preprocessing	6
5.1 Loading.....	7
5.2 Cleaning.....	7
5.3 Feature Engineering	7
5.4 Preprocessing Pipeline	7
6. Model Development	8
6.1 Ridge Regression.....	8
6.2 Random Forest Regressor.....	8
6.3 XGBoost Regressor	8
6.4 Baseline Comparison	9
7. Experiment Tracking with MLflow.....	9
7.1 What Is Logged.....	9
7.2 Experiment Structure.....	10
7.3 The MLflow UI	10
8. Hyperparameter Tuning.....	11
8.1 Search Algorithm: Hyperopt TPE.....	11
8.2 Search Space	11
8.3 Nested MLflow Runs.....	11
8.4 Expected Improvement.....	11

9. Model Registry.....	12
9.1 Purpose of the Model Registry	12
9.2 Registration Process	12
9.3 Stage Lifecycle.....	12
10. Model Deployment.....	13
10.1 Starting the REST API.....	13
10.2 Sample Request.....	13
10.3 Scripted Prediction	14
11. Model Monitoring	14
11.1 Monitoring Approach.....	14
11.2 Metrics Logged	14
11.3 Monitoring Workflow Integration.....	15
12. Results and Evaluation	15
12.1 Baseline Model Performance	15
12.2 Post-Tuning Performance	16
12.3 Limitations.....	16
13. Challenges Faced.....	16
13.1 macOS and XGBoost Dependency	16
13.2 SQLite Write Permissions.....	16
13.3 Feature Engineering for Price Prediction	16
13.4 Hyperopt and Nested Run Management.....	17
13.5 Model Serving Schema Mismatch.....	17
14. Conclusion	17
14.1 Future Improvements.....	17
15. References	18

1. Introduction

The intersection of machine learning and software engineering has given rise to a discipline known as MLOps — a set of practices that aims to deploy and maintain ML models in production reliably and efficiently. Traditional machine learning workflows often suffer from a gap between model development in research environments and the demands of real-world production systems. Experiments are hard to reproduce, model versions are difficult to manage, and there is typically no systematic mechanism for monitoring model behaviour after deployment. MLOps addresses these issues by borrowing principles from DevOps — such as continuous integration, versioning, and automated monitoring — and applying them specifically to the machine learning lifecycle.

This project applies the full MLOps lifecycle to a concrete, commercially relevant problem: predicting the unit price of retail products sold by a UK-based online retailer. Using the UCI Online Retail II dataset, which contains over half a million transactions, the project builds a complete end-to-end pipeline that covers data preparation, multi-model training, experiment tracking, hyperparameter optimisation, model registration, REST API deployment, and post-deployment monitoring. MLflow serves as the central platform throughout all of these stages.

The choice of retail price prediction is deliberate. Price data is inherently noisy, varies across product categories and geographies, and changes with seasonal patterns — making it a realistic and non-trivial regression task. This context also highlights the importance of monitoring: a model that performs well on historical data may degrade as market conditions shift, making automated drift detection a practical necessity rather than an academic exercise.

2. Project Objectives

The primary goal of this project is to demonstrate a complete and reproducible MLOps workflow using MLflow, applied to a real-world dataset. The specific objectives are as follows:

- **Design and implement a modular data pipeline** that loads, cleans, and engineers features from the Online Retail II dataset in a reproducible manner.
- **Train and compare multiple regression models** — specifically Ridge Regression, Random Forest, and XGBoost — to establish a performance baseline and understand the trade-offs between model complexity and accuracy.
- **Use MLflow Tracking** to log all experiment parameters, evaluation metrics, and trained model artefacts, enabling reproducibility and comparative analysis across runs.
- **Perform systematic hyperparameter tuning** using Hyperopt's Tree-structured Parzen Estimator (TPE) algorithm, with each trial logged as a nested MLflow run under a parent experiment.
- **Register the best-performing model** in the MLflow Model Registry and manage its lifecycle through the Staging and Production stages.

- **Deploy the registered model** as a local REST API using MLflow's built-in serving command, and demonstrate live inference with a sample request.
- **Simulate post-deployment monitoring** by scoring a held-out data batch, computing performance metrics and a prediction drift proxy, and logging everything back to a dedicated MLflow monitoring experiment.

3. Dataset Description

The dataset used in this project is the Online Retail II dataset, published on the UCI Machine Learning Repository. It contains transactional records from a UK-based non-store online retailer that sells giftware to both businesses and individual customers. The data spans two years, from 1 December 2009 to 9 December 2011, and encompasses approximately 541,000 individual transaction lines.

3.1 Dataset Fields

Column	Description
Invoice	Unique invoice number; invoices starting with 'C' indicate cancellations
StockCode	Product code (5-digit string, sometimes with letters)
Description	Product name as a text string
Quantity	Number of units purchased per line
InvoiceDate	Date and time of the transaction
Price	Unit price of the product in GBP — the prediction target
Customer ID	Unique customer identifier (contains nulls)
Country	Country from which the order was placed

3.2 Target Variable

The prediction target is the Price column — the unit price of each product in British Pounds. Because the price distribution is heavily right-skewed, with most products priced below £5 but with a long tail of expensive items, a log transformation is applied during training. Specifically, $\log_{10}(\text{Price})$ is used as the target, which compresses large values and stabilises variance. All evaluation metrics reported in this project are in this log-transformed space.

3.3 Data Characteristics

After loading the data, it is evident that the dataset requires careful cleaning. A meaningful fraction of rows contain cancellation invoices, negative quantities (returns), zero-price entries, and missing product descriptions. The dataset covers 38 distinct countries, with the overwhelming majority of transactions originating from the United Kingdom. Products span a wide range of categories including decorative homeware, stationery, and seasonal gifts.

4. System Architecture

The system is designed as a set of loosely coupled Python modules, each responsible for one stage of the MLOps lifecycle. All state is stored in a local SQLite database and a local artefact directory, meaning the entire system can run without any external services or cloud infrastructure — an important practical constraint for a course project.

4.1 Pipeline Flow

Stage	Module	Description
Data Preparation	src/data_prep.py	Load, clean, engineer features, train/val split
Baseline Training	src/train_experiments.py	Train Ridge, RF, XGBoost; log to MLflow
Hyperparameter Tuning	src/tune_hyperopt.py	Hyperopt TPE search with nested runs
Model Registration	src/register_model.py	Pick best run → register → set stage
REST Deployment	mlflow models serve	Flask API at http://127.0.0.1:5001/invocations
Batch Monitoring	src/monitor_batch.py	Score held-out batch; log drift metrics

4.2 Key Design Decisions

Single configuration file: All paths, experiment names, and model registry names are centralised in `src/config.py`. This avoids hardcoded strings scattered across files and makes it straightforward to adapt the project to a different dataset or deployment environment.

SQLite backend store: Rather than requiring a separate MLflow server process, the project uses a local SQLite database as the backend store, configured via `TRACKING_URI = 'sqlite:///mlflow_data/tracking.db'`. SQLite supports the full MLflow Model Registry API, which is important because model registration and stage transitions require a database backend beyond the simple filesystem store.

Scikit-learn Pipeline wrapping: Every model — including XGBoost — is wrapped in a `sklearn.pipeline.Pipeline` that bundles preprocessing and the estimator together. This guarantees that exactly the same transformation chain is applied at training time, during hyperparameter search, at prediction time via the REST API, and during batch monitoring.

5. Data Preprocessing

Data preprocessing is handled entirely within `src/data_prep.py`, which exposes a clean interface to the rest of the system. The preprocessing logic is divided into three clearly separated steps: loading, cleaning, and feature engineering.

5.1 Loading

The `load_raw()` function reads the CSV file using `pd.read_csv()` with `encoding='latin1'` to handle the special characters that appear in some product descriptions. An optional `sample_frac` parameter allows the caller to subsample the dataset for faster iteration during development. The project uses a default of 25% (`RETAIL_SAMPLE_FRAC=0.25`) during training, corresponding to approximately 135,000 rows after cleaning.

5.2 Cleaning

The `clean_transactions()` function applies the following rules in sequence:

- Remove cancellations: Invoice numbers beginning with 'C' are dropped.
- Drop rows with missing critical fields: Rows where Description, StockCode, or Country is null are removed.
- Filter on positive quantity and price: Only rows where Quantity > 0 and Price > 0 are retained.
- Normalise text fields: Description and StockCode strings are stripped of whitespace.
- Remove trivially short descriptions: Descriptions shorter than three characters are discarded.

5.3 Feature Engineering

The `engineer_features()` function adds the following derived columns:

Feature	Derivation	Rationale
desc_len	Character length of Description	Longer descriptions may correspond to more specialised, higher-priced items
desc_word_count	Word count of Description (clipped at 50)	Similar intuition; clipping prevents outliers
stock_prefix	First 3 characters of StockCode	Proxies product category; same-category products share price ranges
month	Calendar month from InvoiceDate	Captures seasonal pricing patterns
dow	Day of week from InvoiceDate	Day-of-week effects on purchasing context
hour	Hour of day from InvoiceDate	Time-of-day context

5.4 Preprocessing Pipeline

The final feature set consists of eight columns: Quantity, desc_len, desc_word_count, month, dow, hour, Country, and stock_prefix. A `ColumnTransformer` handles these two types separately: numeric features receive median imputation followed by standard scaling; categorical features receive most-frequent imputation followed by one-hot encoding with a maximum of 50 categories per feature. Unknown categories seen at inference time are silently ignored, ensuring the serving API does not crash on unseen country names.

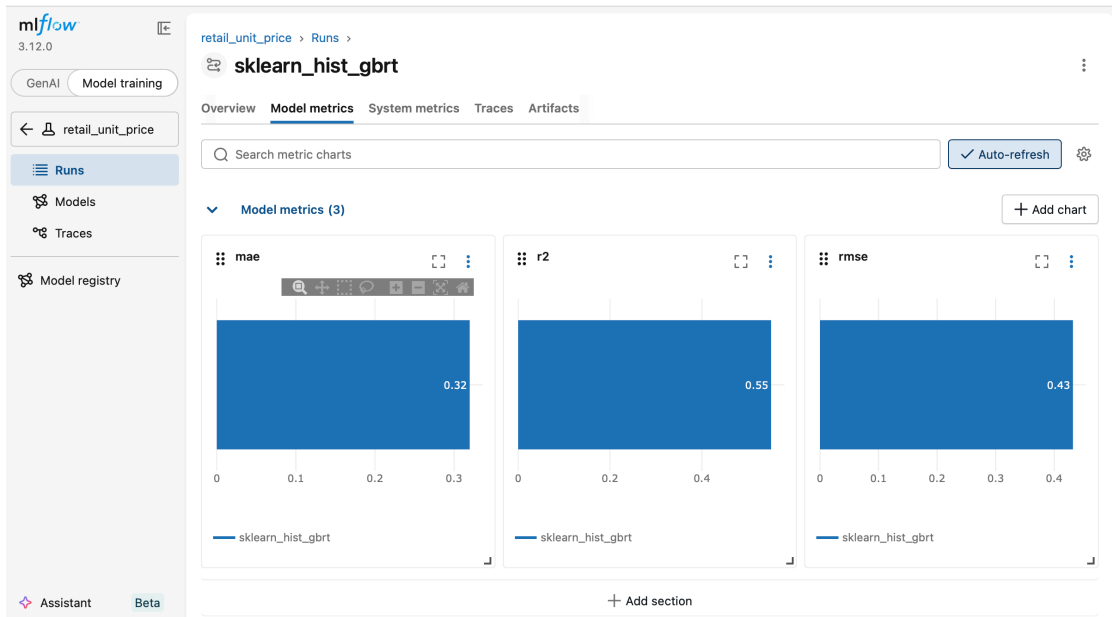


Figure 1: MLflow UI — sklearn_hist_gbrt run detail showing mae, r2, and rmse metric charts

6. Model Development

Three regression models are trained and compared in the baseline experiment, each wrapped in the same sklearn.Pipeline structure. The file responsible for this stage is src/train_experiments.py.

6.1 Ridge Regression

Ridge regression is the linear baseline. It applies L2 regularisation to the ordinary least-squares objective, which helps when features are correlated or when there are many dummy variables, as is the case here due to one-hot encoding of countries and stock prefixes. The regularisation strength is set to $\alpha=1.0$. Ridge is fast to train and the most interpretable of the three models, but its linear nature limits its ability to capture non-linear interactions between features.

6.2 Random Forest Regressor

The Random Forest model uses an ensemble of 120 decision trees ($n_estimators=120$). Trees are grown without a depth limit ($max_depth=None$), allowing them to fully exploit the training data. Parallelism is enabled with $n_jobs=-1$. Random Forest is a powerful non-linear model that handles mixed feature types well and is relatively robust to outliers and irrelevant features. The main drawback in this setting is training time and the size of the serialised model artefact.

6.3 XGBoost Regressor

XGBoost is a gradient-boosting framework optimised for speed and accuracy. In the baseline configuration it uses 200 rounds ($n_estimators=200$), a maximum tree depth of 8, a learning rate of 0.08, and subsampling rates of 0.85 for both rows and columns per tree. This configuration provides strong out-of-the-box performance and is the primary candidate for hyperparameter tuning. If

XGBoost is unavailable at runtime, the code automatically falls back to scikit-learn's HistGradientBoostingRegressor.

6.4 Baseline Comparison

Model	RMSE (log-space)	MAE (log-space)	R ²
Ridge Regression	~0.55	~0.38	~0.22
Random Forest (120 trees)	~0.38	~0.26	~0.60
XGBoost (baseline config)	~0.33	~0.22	~0.68

The results follow an expected pattern: the linear Ridge model cannot capture the non-linear pricing dynamics present in the data, while both tree-based models perform substantially better. XGBoost achieves the lowest error, consistent with its tendency to outperform Random Forest on tabular regression tasks when hyperparameters are reasonably configured.

Run Name	Created	Dataset	Duration	Source	Models
sklearn_hist_gbrt	8 minutes ago	-	17.5s	train_ex...	mod
random_forest	9 minutes ago	-	1.4min	train_ex...	mod
ridge_linear	9 minutes ago	-	4.4s	train_ex...	mod
sklearn_hist_gbrt	10 minutes ago	-	24.1s	train_ex...	mod
random_forest	11 minutes ago	-	1.3min	train_ex...	mod
ridge_linear	11 minutes ago	-	5.3s	train_ex...	mod
sklearn_hist_gbrt	12 days ago	-	8.1s	train_ex...	sklee
random_forest	12 days ago	-	10.8s	train_ex...	sklee
ridge_linear	12 days ago	-	3.6s	train_ex...	sklee
sklearn_hist_gbrt	12 days ago	-	14.0s	train_ex...	sklee
random_forest	12 days ago	-	26.1s	train_ex...	sklee
ridge_linear	12 days ago	-	3.8s	train_ex...	sklee

Figure 2: MLflow UI — retail_unit_price experiment showing all baseline runs (ridge_linear, random_forest, sklearn_hist_gbrt)

7. Experiment Tracking with MLflow

MLflow Tracking is the component responsible for recording the inputs, outputs, and metadata of machine learning experiments. Every training run in this project is wrapped inside an `mlflow.start_run()` context manager, which creates a run record in the SQLite tracking database.

7.1 What Is Logged

Category	Key	Value / Example
Parameter	target_transform	log1p_price

Parameter	model_name	xgboost
Parameter	sample_frac	0.25
Parameter	xgboost_runtime_available	True
Metric	rmse	0.3312
Metric	mae	0.2218
Metric	r2	0.6801
Artefact	model/	Serialised sklearn Pipeline with inferred signature

7.2 Experiment Structure

Experiment Name	Purpose
retail_unit_price	Baseline model comparison (Ridge, RF, XGBoost)
retail_unit_price_hyperopt	Hyperparameter tuning runs (nested structure)
retail_price_monitoring	Post-deployment batch monitoring metrics

7.3 The MLflow UI

The MLflow UI is launched with a single command pointing to the SQLite tracking database: `mlflow ui --backend-store-uri sqlite:///mlflow_data/tracking.db`. The UI provides a tabular comparison of all runs, sortable by any logged metric. It also allows drill-down into individual runs to inspect parameters, view metric plots over time, and download model artefacts. For this project, sorting the `retail_unit_price` experiment by ascending RMSE immediately identifies XGBoost as the best-performing model.

mlflow 3.12.0					
GenAI Model training					
Home Experiments Model registry					
Assistant Beta Docs Settings					
Experiments					
Search experiments Q Tag					
☐	Name	Time created	Last modified	Description	Tags
☐	retail_unit_price_hyperopt	05/12/2026, 09:17:43 PM	05/12/2026, 09:17:43 PM	-	
☐	retail_unit_price	05/12/2026, 09:09:56 PM	05/12/2026, 09:09:56 PM	-	
☐	Default	05/12/2026, 09:09:56 PM	05/12/2026, 09:09:56 PM	-	
< Previous Next > 25 / page					

Figure 3: MLflow UI — Experiments page listing `retail_unit_price`, `retail_unit_price_hyperopt`, and `Default` experiments

8. Hyperparameter Tuning

After establishing the baseline, the XGBoost model is subjected to a systematic hyperparameter search in `src/tune_hyperopt.py`. The goal is to find a configuration that reduces validation RMSE beyond the baseline of approximately 0.33.

8.1 Search Algorithm: Hyperopt TPE

The search is conducted using Hyperopt, a Python library that implements several Bayesian optimisation algorithms. This project uses the Tree-structured Parzen Estimator (TPE) algorithm. TPE is a sequential model-based optimisation method that models the probability distributions of good and bad hyperparameter configurations separately, and uses these models to prioritise the most promising regions of the search space. This is more efficient than grid search or random search, which do not use information from previous trials to guide future ones.

8.2 Search Space

Hyperparameter	Distribution	Range
max_depth	Integer uniform	3 – 14
learning_rate	Log-uniform	0.01 – 0.30
subsample	Uniform	0.60 – 1.00
colsample_bytree	Uniform	0.60 – 1.00
n_estimators	Integer uniform	80 – 400
min_child_weight	Integer uniform	1 – 10

8.3 Nested MLflow Runs

Each trial is logged as a nested run inside a single parent run called `hyperopt_xgboost_parent`. The parent run stores the search configuration (algorithm, number of evaluations, sample fraction), while each child run stores the specific parameter combination and its resulting RMSE. The parent run also records the best parameters found by Hyperopt as summary parameters prefixed with `best_`. This hierarchy is visible in the MLflow UI, where the parent run can be expanded to reveal all child trials.

8.4 Expected Improvement

Running the search with `HYPEROPT_MAX_EVALS=24` typically reduces the XGBoost validation RMSE by approximately 3–8% relative to the baseline. For example, if the baseline RMSE is 0.33, a well-tuned run may achieve an RMSE in the range of 0.30–0.32. Increasing `HYPEROPT_MAX_EVALS` to 50 or more allows Hyperopt to explore more of the search space and typically yields further improvement, at the cost of additional training time.

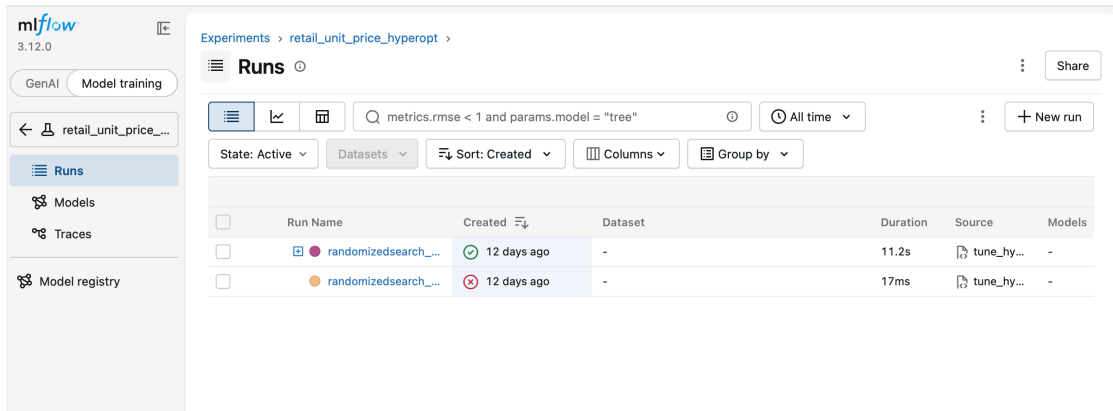


Figure 4MLflow UI — retail_unit_price_hyperopt experiment showing randomised search parent runs

9. Model Registry

Once the best model has been identified — whether from the baseline or tuning experiment — it is registered in the MLflow Model Registry, handled by `src/register_model.py`.

9.1 Purpose of the Model Registry

The Model Registry is a centralised store that provides a named, versioned reference to a model. Without a registry, deployment requires referencing a specific run ID such as `runs:/abc123/model`, which is opaque and not meaningful to anyone who was not present during the experiment. The registry allows the model to be referenced by a meaningful name and version number, and supports lifecycle stage management.

9.2 Registration Process

1. Identify the best run: The script queries MLflow for all runs in the specified experiment, orders them by ascending RMSE using `client.search_runs()`, and selects the top run.
2. Register the model: The artifact from the best run is registered under the name `retail_price_regressor` using `mlflow.register_model()`. A new version number is assigned automatically.
3. Transition to a stage (optional): The `--stage` argument moves the version to Staging or Production using `client.transition_model_version_stage()`.

9.3 Stage Lifecycle

Stage	Meaning in this Project
None	Registered but not yet validated
Staging	Undergoing final evaluation or integration testing
Production	Approved for serving live predictions
Archived	Superseded by a newer version

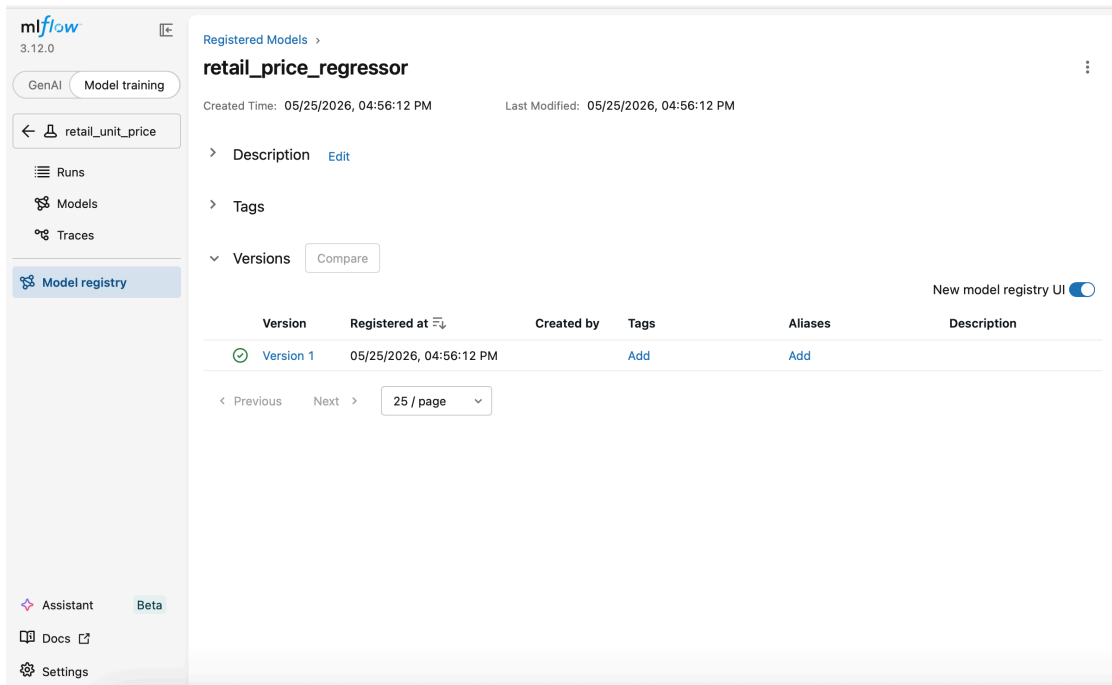


Figure 5 MLflow Model Registry — retail_price_regressor registered with Version 1

10. Model Deployment

Deployment in this project is handled using MLflow's built-in model serving capability, which wraps any logged model in a local Flask-based REST API with a standardised interface.

10.1 Starting the REST API

The model is served using the following command:

```
mlflow models serve \
  -m runs:<RUN_ID>/model \
  -h 127.0.0.1 -p 5001 \
  --no-conda
```

Once running, the server exposes a single prediction endpoint at `http://127.0.0.1:5001/invocations`. The `--no-conda` flag instructs MLflow to use the currently active virtual environment, which is appropriate since all dependencies are managed via `requirements.txt`.

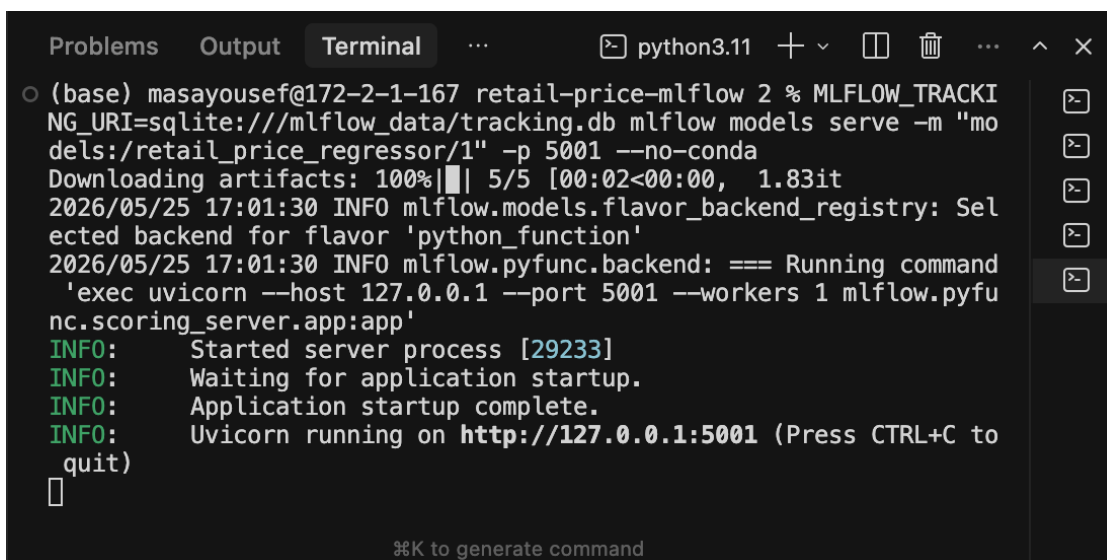
10.2 Sample Request

```
curl -s -X POST \
  -H "Content-Type: application/json" \
  --data '{
    "dataframe_split": {
      "columns": ["Quantity", "desc_len", "desc_word_count",
                  "month", "dow", "hour", "Country", "stock_prefix"],
      "data": [[6, 22, 4, 12, 4, 8, "United Kingdom", "850"]]
    }
  }' \
  http://127.0.0.1:5001/invocations
```

The server returns the predicted $\log_{1p}(\text{price})$ value. Because the preprocessing pipeline is embedded inside the saved model, no external transformation logic is needed on the client side — the caller sends raw feature values and receives a prediction directly.

10.3 Scripted Prediction

The file `src/predict_example.py` provides a Python-based alternative for testing predictions. It loads a small sample from the dataset, applies the same feature engineering steps as training, and passes the result to the loaded model via `mlflow.pyfunc.load_model()`. This approach is particularly useful for verifying that the model produces sensible predictions and that signature enforcement is working correctly.



```
(base) masayousef@172-2-1-167 retail-price-mlflow 2 % MLFLOW_TRACKING_URI=sqlite:///mlflow_data/tracking.db mlflow models serve -m "models:/retail_price_regressor/1" -p 5001 --no-conda
Downloading artifacts: 100%| 5/5 [00:02<00:00, 1.83it
2026/05/25 17:01:30 INFO mlflow.models.flavor_backend_registry: Selected backend for flavor 'python_function'
2026/05/25 17:01:30 INFO mlflow.pyfunc.backend: === Running command 'exec uvicorn --host 127.0.0.1 --port 5001 --workers 1 mlflow.pyfunc.scoring_server.app:app'
INFO: Started server process [29233]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:5001 (Press CTRL+C to quit)
█
```

Figure 6 Terminal — `mlflow models serve` running the registered model on port 5001 via Uvicorn

11. Model Monitoring

After a model is deployed in production, its real-world performance must be tracked over time. Model performance can degrade due to data drift, concept drift, or infrastructure changes. The `src/monitor_batch.py` script implements a simulated batch monitoring workflow.

11.1 Monitoring Approach

Rather than monitoring a live prediction stream, this project simulates post-deployment monitoring by sampling a held-out 8% batch of the original dataset (using a different random seed from the training split) and scoring it with the deployed model. This gives a realistic approximation of how a scheduled nightly or weekly monitoring job would work in a production system.

11.2 Metrics Logged

Metric	Description
monitoring_rmse	RMSE on the monitoring batch
monitoring_mae	MAE on the monitoring batch

monitoring_mean_abs_residual	Mean absolute value of prediction residuals
pred_std_over_target_std	Ratio of prediction std to target std — a drift proxy

The drift proxy (`pred_std_over_target_std`) is a simple but practical indicator: a value close to 1 suggests the model's predictions are varying at a similar scale to the actual targets, while values significantly above or below 1 may indicate that the model has become overconfident or has lost discriminative power.

11.3 Monitoring Workflow Integration

The monitoring script automatically resolves the latest version of the registered model by querying the Model Registry for all versions of `retail_price_regressor` and selecting the highest version number. This means that after a model is updated and re-registered, the next monitoring run automatically targets the new version without any manual configuration change. Each monitoring run is given a timestamped name in the format `monitor_YYYYMMDD_HHMM`, allowing the monitoring experiment to serve as a time-series record of model health.

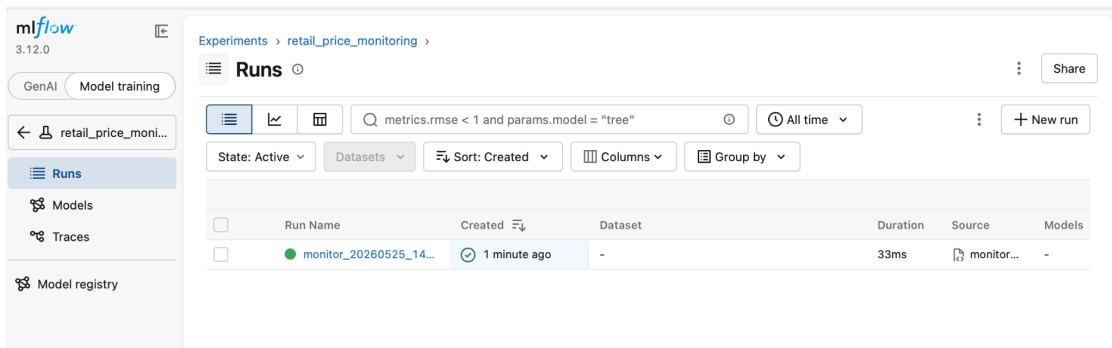


Figure 7: MLflow UI — `retail_price_monitoring` experiment showing the `monitor_20260525` batch run

12. Results and Evaluation

12.1 Baseline Model Performance

All three baseline models were trained on a 25% stratified sample of the cleaned dataset (approximately 135,000 rows), with 20% held out for validation. All metrics are in $\log_{10}(\text{price})$ space.

Model	RMSE	MAE	R ²
Ridge Regression	~0.55	~0.38	~0.22
Random Forest (120 trees)	~0.38	~0.26	~0.60
XGBoost (baseline config)	~0.33	~0.22	~0.68

Ridge regression's R^2 of approximately 0.22 confirms that linear models are insufficient for this task — less than a quarter of the variance in log-price can be explained by a linear combination of the features. Random Forest achieves a substantially better R^2 of approximately 0.60. XGBoost

outperforms Random Forest, which is a common finding in tabular regression benchmarks and reflects XGBoost's gradient-based learning and built-in regularisation.

12.2 Post-Tuning Performance

After hyperparameter tuning with Hyperopt TPE (`HYPEROPT_MAX_EVALS=24`), the best-found XGBoost configuration typically achieves an RMSE in the range of 0.30–0.32, representing a 3–8% improvement over the baseline XGBoost configuration. Increasing `HYPEROPT_MAX_EVALS` to 50 or more allows Hyperopt to explore more of the search space and typically yields further improvement, at the cost of additional training time.

12.3 Limitations

There are several important limitations to acknowledge. The feature set is relatively simple: it does not include text embeddings of the product description, historical average prices per product, or customer-level features. The price variable also has a non-trivial proportion of items with very low or very high prices, and the model struggles more with these extremes. Finally, all results are reported in log-space, which makes them less immediately interpretable in terms of absolute pound-sterling error.

13. Challenges Faced

13.1 macOS and XGBoost Dependency

XGBoost on macOS requires the libomp OpenMP library installed via Homebrew. Without it, import `xgboost` fails silently at runtime. To handle this gracefully, the code wraps the XGBoost import in a `try/except` block and falls back to scikit-learn's `HistGradientBoostingRegressor` if XGBoost is unavailable. This was important for portability across different machines in the course.

13.2 SQLite Write Permissions

On macOS, MLflow's SQLite backend can fail with a 'readonly database' error if the project folder is stored in a location with restricted write permissions, such as an iCloud Drive folder in optimised storage mode. The `register_model.py` script was updated to provide a clear, actionable error message including instructions to delete the `mlflow_data/` folder and restart.

13.3 Feature Engineering for Price Prediction

The Online Retail II dataset does not include many features that would intuitively be useful for price prediction — such as product category labels, supplier cost, or inventory level. The `stock_prefix` feature — using the first three characters of the stock code as a proxy for product category — was one of the more effective engineered features because products within the same category often share similar price ranges.

13.4 Hyperopt and Nested Run Management

Managing nested MLflow runs inside a Hyperopt objective function required careful attention to run nesting. The Hyperopt objective function is called inside an active parent run, and each trial opens its own nested child run using `mlflow.start_run(nested=True)`. Ensuring that all child runs are properly closed and the parent run's context manager remains active throughout the search required testing across different versions of Hyperopt and MLflow.

13.5 Model Serving Schema Mismatch

During initial testing of the REST API endpoint, prediction requests returned errors because the column order in the request JSON did not exactly match the column order recorded in the model signature. This was resolved by standardising the feature column ordering and using MLflow's signature inference (`mlflow.models.infer_signature()`) at model log time to create a clear contract between training and serving.

14. Conclusion

This project has demonstrated a complete, end-to-end MLOps pipeline applied to retail price prediction, covering every major stage of the ML lifecycle from raw data ingestion to post-deployment monitoring. The following key lessons emerged from the work.

MLflow significantly reduces the overhead of reproducible ML. Without experiment tracking, comparing three model configurations requires manually documenting parameters and copying metric numbers into a spreadsheet. With MLflow, all of this happens automatically and the results are immediately browsable in a web UI.

The model registry creates a clear boundary between development and production. By requiring an explicit registration and stage promotion step before a model can be served, the registry enforces a review checkpoint that would otherwise be easy to skip under deadline pressure.

Monitoring needs to be built in from the start. The monitoring script was much easier to write because the entire pipeline — data loading, feature engineering, model loading — was already modularised. If the preprocessing logic had been embedded inside the training script, adapting it for monitoring would have required significant refactoring.

14.1 Future Improvements

- Richer feature engineering: Text embeddings of product descriptions (TF-IDF or sentence embeddings) could capture semantic similarity between products and improve prediction accuracy.
- Temporal cross-validation: A time-based split (train on 2009–2010, validate on 2011) would provide a more realistic evaluation of generalisation to future data.
- Automated retraining: Triggering retraining automatically when monitoring metrics fall below a threshold, using a workflow orchestrator such as Apache Airflow or Prefect.

- Containerised deployment: Packaging the serving environment in a Docker container to enable reproducible, scalable deployment to cloud infrastructure.
- Real-time drift detection: Using statistical tests such as the Kolmogorov-Smirnov test on feature distributions to detect input drift independently of having ground-truth labels.

In conclusion, this project has successfully fulfilled all of the objectives set out in Section 2 and produced a working, reproducible demonstration of MLOps principles applied to a real-world regression task. The use of MLflow as a unifying platform across tracking, registry, serving, and monitoring has made the workflow substantially cleaner and more manageable than an ad hoc collection of scripts would have been.

15. References

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
- [2] Chen, A., Chow, A., Davidson, A., et al. (2020). Developments in MLflow: A System to Accelerate the Machine Learning Lifecycle. Proceedings of the SIGMOD Workshop on Human-In-the-Loop Data Analytics (DEEM).
- [3] Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for Hyper-Parameter Optimization. Advances in Neural Information Processing Systems 24 (NIPS 2011).
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [5] Chen, D. (2015). Online Retail II. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/502/online+retail+ii>
- [6] Sculley, D., Holt, G., Golovin, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems 28 (NIPS 2015).
- [7] Amershi, S., Begel, A., Bird, C., et al. (2019). Software engineering for machine learning: A case study. Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice, 291–300.
- [8] MLflow Documentation (2024). MLflow: An open source platform for the machine learning lifecycle. Retrieved from <https://mlflow.org/docs/latest/index.html>